

Excepciones y Validación

1. Gestión de Excepciones y Robustez del Software

1.1. Concepto y Definición de Excepción

En el desarrollo de software, una **excepción** es un evento no deseado o inesperado que ocurre durante el tiempo de ejecución de un programa e interrumpe el flujo normal de las instrucciones. A diferencia de los errores de sintaxis, que impiden la compilación, las excepciones surgen cuando el programa ya está corriendo debido a factores internos (como una división por cero) o externos (como un archivo no encontrado o una pérdida de conexión de red).

Cuando ocurre un evento excepcional dentro de un método, el sistema crea un **objeto de excepción**. Este objeto encapsula información técnica crucial: el nombre y descripción del error, el estado del programa en ese instante y la traza de la pila de llamadas (*stack trace*). El manejo de excepciones es el proceso de dar respuesta a estos eventos para evitar que la aplicación finalice de forma abrupta y permitir, si es posible, que recupere su funcionamiento normal.

1.2. La Jerarquía de Clases Throwable

Java implementa el manejo de errores mediante una estructura jerárquica de clases basada en la herencia. Todas las clases que representan errores o excepciones derivan de la superclase **java.lang.Throwable**. Esta clase madre provee atributos fundamentales como el mensaje descriptivo y la causa (una referencia a otra excepción que originó el problema), además de métodos esenciales como `printStackTrace()` para depuración.

De `Throwable` se desprenden dos ramas principales que separan los problemas según su gravedad y origen:

- **Error**: Representa condiciones críticas e irreversibles del entorno de ejecución o de la Máquina Virtual de Java (JVM). Ejemplos comunes son `OutOfMemoryError` o `StackOverflowError`. El programador no debe intentar capturar ni manejar estos errores, ya que trascienden la lógica de la aplicación.

- **Exception:** Representa condiciones que una aplicación debería intentar capturar y gestionar. Es la rama donde se sitúan los errores lógicos de software que el desarrollador puede prever y tratar para mantener la estabilidad del sistema.

1.3. Excepciones Comprobadas (Checked) vs. No Comprobadas (Unchecked)

Dentro de la clase `Exception`, Java establece una distinción fundamental que afecta la forma en que escribimos el código:

1. **Excepciones No Comprobadas (Unchecked):** Son aquellas que heredan de `RuntimeException`. Representan errores de programación o lógica, como acceder a un índice fuera de rango (`IndexOutOfBoundsException`) o invocar métodos sobre una referencia nula (`NullPointerException`). El compilador no obliga a capturarlas ni a declararlas en la firma del método, ya que se espera que el programador corrija la lógica para evitarlas en lugar de simplemente "tratarlas".
2. **Excepciones Comprobadas (Checked):** Son todas aquellas que heredan de `Exception` pero no de `RuntimeException`. Suelen deberse a factores externos fuera del control directo del programador, como problemas de entrada/salida (`IOException`) o errores en bases de datos (`SQLException`). Java obliga explícitamente a que estas excepciones sean capturadas mediante un bloque `try-catch` o declaradas en la firma del método con la cláusula `throws`.

1.4. El Mecanismo try-catch-finally

Para gestionar excepciones, Java provee una estructura de bloques específica que permite separar la lógica de negocio del código de tratamiento de errores.

- **try:** Encierra el conjunto de instrucciones propensas a fallar o que dependen del éxito de una operación anterior. Si ocurre una excepción dentro de este bloque, la ejecución se detiene inmediatamente y el control salta al manejador correspondiente.
- **catch:** Es el bloque encargado de capturar y tratar la excepción. Puede haber múltiples bloques `catch` para manejar distintos tipos de excepciones de forma discriminada (Multi-catch). Es vital ordenarlos de lo más específico a lo más general para que el polimorfismo no oculte errores particulares.
- **finally:** Este bloque es opcional y se utiliza para ejecutar código que debe correr siempre, independientemente de si se lanzó una excepción o si hubo un `return` en los bloques anteriores. Su uso principal es la liberación de recursos externos.

Ejemplo de sintaxis básica:

```

try {
    int resultado = 10 / 0;           // Lanza ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Error: No se puede dividir por cero.");
} finally {
    System.out.println("Operación finalizada.");
}

```

1.5. Propagación de Excepciones: throw y throws

No siempre es adecuado tratar una excepción en el mismo lugar donde ocurre. Java permite delegar esta responsabilidad mediante dos palabras clave:

- **throw**: Se utiliza dentro de un método para lanzar una instancia de una excepción de forma explícita. Cuando se ejecuta, el flujo del método se interrumpe inmediatamente. Se suele usar para validar requisitos de un método (por ejemplo, evitar años negativos).
- **throws**: Se coloca en la firma del método para anunciar que este puede propagar ciertas excepciones hacia quien lo invoque. Es obligatorio para excepciones comprobadas que no son tratadas internamente.

Ejemplo de propagación:

```

public void procesarArchivo(String ruta) throws IOException {
    if (ruta == null) {
        throw new IllegalArgumentException("La ruta no puede ser nula");
    }
    // instrucciones que podrían haber lanzado una IOException
}

```

1.6. Gestión Avanzada: try-with-resources

A partir de Java 7, se introdujo la sentencia **try-with-resources** para simplificar la gestión de recursos que implementan la interfaz `AutoCloseable` (como archivos o conexiones). Esta sintaxis garantiza que el recurso se cerrará automáticamente al finalizar el bloque, evitando fugas de memoria y eliminando la necesidad de bloques `finally` verbosísticos para invocar el método `close()`.

Ejemplo comparativo:

```

// Forma moderna con try-with-resources
try (BufferedReader br = new BufferedReader(new FileReader("test.txt"))) {

```

```
        System.out.println(br.readLine());
    } catch (IOException e) {
        System.out.println("Error al leer archivo: " + e.getMessage());
    }
```

1.7. Creación de Excepciones Personalizadas

Cuando las excepciones estándar de Java no describen con suficiente precisión un error del dominio del negocio, es recomendable crear clases de excepción propias. Esto mejora la semántica y legibilidad del código.

Para crear una, se debe:

1. Definir una clase que extienda de `Exception` (para checked) o `RuntimeException` (para unchecked).
2. Por convención, el nombre debe terminar en la palabra `Exception`.
3. Implementar constructores que invoquen a la superclase mediante `super()`, permitiendo pasar mensajes o causas.

```
public class SaldoInsuficienteException extends Exception {
    public SaldoInsuficienteException(String mensaje) {
        super(mensaje);
    }
}
```

2. Validación de Datos y Entrada/Salida (E/S)

2.1. Estrategias de Validación de Entradas

La validación es el proceso de asegurar que los datos ingresados por el usuario cumplen con los requisitos antes de ser procesados por la lógica de negocio. Una práctica robusta es el uso de **ciclos de validación** (`while` o `do-while`) en lugar de simples estructuras condicionales `if-else`. Los ciclos permiten "encerrar" al usuario hasta que proporcione un valor válido, evitando que el programa termine abruptamente ante un error tipográfico.

Además, es común realizar transformaciones de datos para facilitar la validación, como convertir cadenas a mayúsculas con `toUpperCase()` para comparar categorías sin importar si el usuario usó minúsculas.

2.2. Flujos de Datos (Streams)

En Java, la entrada y salida se basa en el concepto de **flujo (stream)**, que representa una secuencia de datos conectada a una fuente (entrada) o un destino (salida). Los flujos pueden ser:

- **Orientados a Bytes:** Clases `InputStream` y `OutputStream`. Se usan para datos binarios (imágenes, archivos comprimidos).
- **Orientados a Caracteres:** Clases `Reader` y `Writer`. Utilizan la convención Unicode y son ideales para archivos de texto plano.

2.3. Operaciones con Archivos de Texto

Para trabajar con archivos de texto de manera eficiente, se utilizan buffers que reducen el número de accesos directos al disco, mejorando el rendimiento.

- **File:** Permite manipular metadatos del archivo (nombre, ruta, si existe) sin abrirlo para lectura.
- **FileReader / FileWriter:** Abren flujos de caracteres hacia o desde un archivo.
- **BufferedReader / BufferedWriter:** Añaden capacidades de búfer y métodos convenientes como `readLine()` (leer línea completa) y `newline()` (salto de línea).

Ejemplo de escritura en archivo:

```
try (BufferedWriter writer = new BufferedWriter(new FileWriter("notas.txt"))) {
    writer.write("Contenido de estudio");
    writer.newLine();
} catch (IOException e) {
    e.printStackTrace();
}
```

3. Pruebas Unitarias y Calidad de Diseño

3.1. Fundamentos de JUnit 5

Las **pruebas unitarias** son un paso crucial para asegurar que el código es robusto y reducir regresiones en el futuro. En Java, el estándar actual es el framework **JUnit 5**. Una prueba unitaria verifica una unidad mínima de código (generalmente un método) en aislamiento.

Conceptos clave de JUnit 5:

- **@Test:** Anotación que marca un método como un caso de prueba.

- **@BeforeEach:** Se ejecuta antes de cada prueba, ideal para inicializar objetos o limpiar el estado (Setup).
- **@DisplayName:** Permite dar nombres descriptivos a los tests para que sean fáciles de entender en los reportes.

3.2. Aserciones y Validación de Comportamiento

Para verificar si el resultado de un método es el esperado, se utilizan las **aserciones** provistas por la clase `Assertions`:

- `assertEquals(esperado, actual)`: Verifica que dos valores sean iguales.
- `assertTrue(condicion)`: Verifica que una condición sea verdadera.
- `assertNotNull(objeto)`: Asegura que una referencia no sea nula.

3.3. Refactorización: Mejora del Diseño Existente

La **refactorización** es el proceso de mejorar la estructura interna del código sin cambiar su comportamiento externo. Sus pilares son la simplicidad, la legibilidad y el rendimiento.

Técnicas fundamentales de refactorización:

- **Renombrado de variables:** Cambiar nombres crípticos (como `p o dse`) por nombres semánticos que expliquen el propósito del dato (`producto`, `descripcionSinEspacios`).
- **Extracción de métodos:** Tomar un bloque de código complejo dentro de un método largo y convertirlo en un método independiente con un nombre claro. Esto reduce la complejidad y facilita la reutilización.

3.4. Integración de Conceptos POO en el Manejo de Errores

El manejo de excepciones en Java está intrínsecamente ligado a la POO. Al lanzar excepciones, estamos enviando mensajes especiales entre objetos para notificar estados anómalos. Además, el uso de la **herencia** permite agrupar y diferenciar tipos de errores (por ejemplo, todas las `IOException` pueden ser tratadas genéricamente si el contexto lo permite).

Un buen diseño orientado a objetos evita el uso de excepciones para el flujo normal de control. Las excepciones deben reservarse para situaciones verdaderamente excepcionales, manteniendo la lógica habitual limpia y legible. Asimismo, el principio de **encapsulación** sugiere que los métodos deben validar sus parámetros antes de procesarlos, protegiendo el estado interno del objeto mediante el lanzamiento de excepciones como `IllegalArgumentException`.